

Latihan Bab 8 — Queue & Simulasi

TicketCounterSimulation – Analisis & Implementasi

1. Kompleksitas Waktu (Worst Case)

Berikut analisis kompleksitas waktu worst-case untuk setiap operasi dalam class **TicketCounterSimulation**:

Operasi / Metode	Kompleksitas	Penjelasan
<code>__init__()</code>	$O(n)$	Membuat array <code>numAgents</code> agen membutuhkan $O(n)$ iterasi.
<code>run()</code>	$O(m \cdot n)$	Loop <code>numMinutes</code> (m) kali, masing-masing memanggil tiga <code>handle</code> dengan kompleksitas $O(n)$.
<code>_handleArrival()</code>	$O(1)$	Satu operasi <code>enqueue</code> pada <code>queue</code> penumpang — $O(1)$.
<code>_handleBeginService()</code>	$O(n)$	Iterasi atas n agen untuk mencari agen yang bebas; <code>dequeue</code> $O(1)$.
<code>_handleEndService()</code>	$O(n)$	Iterasi atas n agen untuk mengecek apakah layanan selesai.
<code>printResults()</code>	$O(1)$	Kalkulasi dan cetak hasil konstan; <code>len(queue)</code> $O(1)$.

Table 8.1 — Sample results of the ticket counter simulation experiment:

Num Minutes	Num Agents	Average Service	Time Between	Average Wait	Passengers Served	Passengers Remaining
100	2	3	2	2.49	49	2
500	2	3	2	3.91	240	0
1000	2	3	2	10.93	490	14
5000	2	3	2	15.75	2459	6
10000	2	3	2	21.17	4930	18
100	2	4	2	10.60	40	11
500	2	4	2	49.99	200	40
1000	2	4	2	95.72	400	104
5000	2	4	2	475.91	2000	465
10000	2	4	2	949.61	4000	948
100	3	4	2	0.51	51	0
500	3	4	2	0.50	240	0
1000	3	4	2	1.06	501	3
5000	3	4	2	1.14	2465	0
10000	3	4	2	1.21	4948	0

2. Eksekusi Manual — Enqueue Kelipatan 3

Kode:

```
values = Queue()
```

```
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
```

Hasil eksekusi manual:

Nilai i yang memenuhi $i \% 3 == 0$ dalam `range(16)` adalah: 0, 3, 6, 9, 12, 15.

Isi Queue (front → rear): [0, 3, 6, 9, 12, 15]

3. Eksekusi Manual — Enqueue & Dequeue

Kode:

```
values = Queue()
for i in range(16):
    if i % 3 == 0:
        values.enqueue(i)
    elif i % 4 == 0:
        values.dequeue()
```

Trace per iterasi:

i	Kondisi	Aksi	Isi Queue (front → rear)
0	$0 \% 3 == 0$ ✓	<code>enqueue(0)</code>	[0]
1	$1 \% 3 \neq 0, 1 \% 4 \neq 0$	-	[0]
2	$2 \% 3 \neq 0, 2 \% 4 \neq 0$	-	[0]
3	$3 \% 3 == 0$ ✓	<code>enqueue(3)</code>	[0, 3]
4	$4 \% 3 \neq 0, 4 \% 4 == 0$ ✓	<code>dequeue()</code> → hapus 0	[3]
5	$5 \% 3 \neq 0, 5 \% 4 \neq 0$	-	[3]
6	$6 \% 3 == 0$ ✓	<code>enqueue(6)</code>	[3, 6]
7	$7 \% 3 \neq 0, 7 \% 4 \neq 0$	-	[3, 6]
8	$8 \% 3 \neq 0, 8 \% 4 == 0$ ✓	<code>dequeue()</code> → hapus 3	[6]
9	$9 \% 3 == 0$ ✓	<code>enqueue(9)</code>	[6, 9]
10	$10 \% 3 \neq 0, 10 \% 4 \neq 0$	-	[6, 9]
11	$11 \% 3 \neq 0, 11 \% 4 \neq 0$	-	[6, 9]
12	$12 \% 3 == 0$ ✓	<code>enqueue(12)</code>	[6, 9, 12]
13	$13 \% 3 \neq 0, 13 \% 4 \neq 0$	-	[6, 9, 12]
14	$14 \% 3 \neq 0, 14 \% 4 \neq 0$	-	[6, 9, 12]
15	$15 \% 3 == 0$ ✓	<code>enqueue(15)</code>	[6, 9, 12, 15]

Isi Queue Akhir (front → rear): [6, 9, 12, 15]

4. Implementasi Metode TicketCounterSimulation

Implementasi tiga metode yang tersisa:

```
def _handleArrival(self, curTime):
    # Rule #1: Jika penumpang tiba, tambahkan ke antrian
    import random
    if random.random() <= self._arriveProb:
        p = Passenger(curTime)
        self._passengerQ.enqueue(p)
        self._numPassengers += 1

def _handleBeginService(self, curTime):
    # Rule #2: Jika ada agen bebas dan ada penumpang di antrian,
```

```

#         mulai layani penumpang tersebut
for i in range(len(self._theAgents)):
    if self._theAgents[i].isFree():
        if not self._passengerQ.isEmpty():
            passenger = self._passengerQ.dequeue()
            self._totalWaitTime += curTime - passenger.timeArrived()
            self._theAgents[i].startService(passenger, curTime +
self._serviceTime)
            break

def _handleEndService(self, curTime):
    # Rule #3: Jika agen selesai melayani, set agen menjadi bebas
    for i in range(len(self._theAgents)):
        if self._theAgents[i].isFinished(curTime):
            self._theAgents[i].stopService()

```

Ketiga metode di atas mengimplementasikan tiga aturan simulasi: kedatangan penumpang (probabilistik), mulai layanan saat agen bebas, dan akhir layanan saat waktu selesai tercapai.

5. Modifikasi: Satuan Detik (bukan Menit)

Ubah parameter numMinutes menjadi numSeconds dan sesuaikan arriveProb:

```

class TicketCounterSimulation:
    def __init__(self, numAgents, numSeconds, betweenTime, serviceTime):
        self._arriveProb = 1.0 / betweenTime # sama seperti sebelumnya
        self._serviceTime = serviceTime
        self._numSeconds = numSeconds # ganti menit → detik
        ...

    def run(self):
        for curTime in range(self._numSeconds + 1): # iterasi per detik
            self._handleArrival(curTime)
            self._handleBeginService(curTime)
            self._handleEndService(curTime)

```

Contoh tabel eksperimen dengan satuan detik (ekuivalen dengan Tabel 8.1):

Num Seconds	Num Agents	Average Service	Time Between	Average Wait	Passengers Served	Passengers Remaining
6000	2	180	120	149.4	49	2
30000	2	180	120	234.6	240	0
60000	2	180	120	655.8	490	14
6000	3	240	120	30.6	51	0
30000	3	240	120	30.0	240	0
60000	3	240	120	63.6	501	3

Catatan: nilai di atas merupakan contoh ilustratif. Hasil aktual bervariasi karena sifat acak simulasi.

6. Fungsi Membalik Urutan Item dalam Queue

Strategi: gunakan stack (list Python) sebagai struktur data tambahan, hanya menggunakan operasi Queue ADT.

```

def reverse_queue(q):
    """
    Membalik urutan item dalam queue q.
    Hanya menggunakan operasi Queue ADT (enqueue, dequeue, isEmpty).
    Menggunakan stack (list) sebagai struktur data pembantu.
    """
    stack = []

```

```
# Langkah 1: Pindahkan semua item dari queue ke stack
while not q.isEmpty():
    stack.append(q.dequeue())

# Langkah 2: Pindahkan kembali dari stack ke queue (urutan terbalik)
while len(stack) > 0:
    q.enqueue(stack.pop())

return q

# Contoh penggunaan:
# q berisi [1, 2, 3, 4, 5] (front=1, rear=5)
# Setelah reverse_queue(q): [5, 4, 3, 2, 1] (front=5, rear=1)
```

Analisis Kompleksitas:

- Waktu: $O(n)$ — setiap item di-dequeue satu kali dan di-enqueue satu kali.
- Ruang: $O(n)$ — stack menyimpan semua n item sementara.